Trade-off Analysis Between Accuracy and Computational Efficiency in Sarcasm Detection Using DistilBERT and RoBERTa

Falah Razan Hibrizi
School of Computing
Telkom University
Bandung, Indonesia
email address

Supervisor Name
School of Computing
Telkom University
Bandung, Indonesia
email address

***Abstract*—**
***Index Terms*—**

I. Introduction

II. Methods

Three questions guided the experiments. The main one concerns the gap in accuracy and cost between RoBERTa and DistilBERT. The second concerns the parent comment, namely whether it helps and how much each way of supplying it costs. Lastly, partial layer freezing of RoBERTa was tested as another way to reduce training cost, as an alternative to switching to the smaller model.

A. Dataset and Preprocessing

This study uses the balanced version of the Self-Annotated Reddit Corpus (SARC) [1], taken from the Kaggle release that was also used in [2] and [3]. The file has 1,010,826 rows, and each row pairs a Reddit comment with the parent comment it responds to. SARC is not labeled by annotators. Instead, a comment is considered sarcastic when its author marked it with "/s", a convention commonly used on Reddit, and this marker was removed from the text when the corpus was constructed [1].

Preprocessing was kept to a minimum. Previous studies on this corpus converted the text to lowercase, removed punctuation or special characters, and applied lemmatization [2], [3]. These steps were not applied in this study, since capitalization, repeated punctuation, and emoticons such as ";)" can be part of how sarcasm is expressed. The cleaning only targeted formatting noise. HTML entities were decoded twice because some rows had been encoded twice, Markdown links were replaced by their visible text, URLs and HTML tags were removed, and extra whitespace was collapsed. User mentions were left in place. No emoji handling was required, as the corpus contains only ASCII characters. In total, only 2,942 comments (0.29%) were modified by this step.

The number of rows removed at each filtering step is given in Table I. Rows with a missing comment, or with text that became empty after cleaning, were removed first. Duplicate pairs and pairs with conflicting labels were then detected using a normalized key, obtained by lowercasing the text and removing its punctuation. The key was used for matching only; the models were always trained on the original text. The last filter deals with an artifact left by the corpus construction. Removing the "/s" marker left some comments with only a fragment that pointed to it, such as "You forgot the". These fragments were detected with a pattern limited to eight words. The pattern matched 8,599 comments, and 8,580 of them were labeled sarcastic. Keeping these rows would allow a model to predict the label from the fragment itself rather than from sarcasm, so they were removed. The final dataset contains 1,000,864 pairs, of which 49.59% are sarcastic.

TABLE I
Row Filtering and Data Split

Filtering step	Removed	Remaining
Raw pairs	–	1,010,826
Missing comment	55	1,010,771
Empty after cleaning	9	1,010,762
Conflicting labels	142	1,010,620
Duplicate pairs	1,157	1,009,463
"/s" fragments	8,599	1,000,864
Split	**Pairs**	**Sarcastic (%)**
Training	799,459	49.66
Validation	100,276	49.38
Test	101,129	49.27

The data were then split into training, validation, and test sets using seed 42. Instead of assigning individual rows, the split was done per comment group, meaning that all rows whose comments have the same normalized key were placed in the same set. The groups were stratified by label. This was necessary because short replies such as "Source?" and "Yes." appear hundreds of times in the corpus, and a row-level split would put the same reply in both the training and test sets. Since whole groups were assigned, the final proportions are close to, but not exactly, 80/10/10. A check after the split found no comment or pair shared between sets.

B. Models

Two pretrained encoders were fine-tuned using the same training pipeline. RoBERTa-base [4] consists of 12 Transformer layers with a hidden size of 768 and has 124.6 million parameters. Its architecture is the same as BERT, but it was pretrained for longer on a larger amount of text. DistilBERT [5] was obtained from BERT through knowledge distillation. It has the same hidden size but only six layers, giving 65.8 million parameters, or slightly more than half the size of RoBERTa. The cased version of DistilBERT was used so that both models could see capitalization, since the byte-level tokenizer of RoBERTa is already case-sensitive. The same pair of models has been compared on a small SARC subset [2] and on news headlines [6]. Both checkpoints were obtained from the Hugging Face hub.

C. Input Configurations

There are several ways to provide the parent comment to the model, and they differ in cost. This study compares three input configurations.

Comment-only. Only the reply is given to the model. This configuration serves as the baseline, showing how well sarcasm can be detected without any context.

Early fusion. The parent and the comment are concatenated into a single input using the sentence-pair format of each tokenizer. The parent comes first, followed by the separator token of the model and then the comment. A similar concatenation of context and response was applied in [7]. Segment embeddings were not used because DistilBERT does not support them. With both texts in one sequence, each comment token can attend to all parent tokens starting from the first layer. The maximum input length was set to 256 tokens. For longer pairs, the end of the parent was truncated, while the comment was only cut when it exceeded the limit by itself. This happened in 52 training rows for RoBERTa and 54 for DistilBERT. Overall, the parent was truncated in 0.85% of the training pairs for RoBERTa and 0.99% for DistilBERT.

Late fusion. In this configuration, the parent and the comment are encoded separately by the same encoder with shared weights. The final hidden state of the first token of the comment is denoted as h_x, and the corresponding vector of the parent as h_y. Both vectors are combined as follows:

$$u = [\, h_x;\ h_y;\ |h_x - h_y|;\ h_x \odot h_y \,], \tag{1}$$

where $[\cdot\,;\cdot]$ denotes concatenation and $\odot$ denotes element-wise multiplication. The absolute difference and the element-wise product are included to capture the interaction between the two representations. The resulting 3,072-dimensional vector u in (1) is fed into a classification head that follows the default head of each model, except that its input is four times larger. For RoBERTa, the head consists of dropout, a linear layer, tanh, dropout, and a second linear layer. For DistilBERT, it consists of a linear layer, ReLU, dropout, and a second linear layer. Truncation followed the same rule as in early fusion, so both fusion configurations use identical tokens. The wider head adds 1.77 million parameters to each model. RoBERTa increases by 2.36 million parameters in total because, in this setting, its encoder also includes a pooling layer of 0.59 million parameters whose output is not used. However, the main cost of late fusion comes from computation. The encoder has to be run twice for each example, and no attention takes place between the parent and the comment.

D. Partial Layer Freezing

Besides using a smaller model, computation can also be reduced by freezing part of a larger model. In this experiment, the embedding layer and the lower encoder layers were frozen, so that only the upper layers and the classification head were updated. Freezing was applied to early fusion only. This configuration gave the best RoBERTa result in the learning rate search, and unlike late fusion, it needs only one pass through the encoder for each example. The number of frozen layers was 0, 6, 9, and 12 out of 12 for RoBERTa, and 0, 3, and 6 out of 6 for DistilBERT. Partially frozen models were trained with three seeds using the learning rate chosen for fully fine-tuned early fusion, without a separate search. When all encoder layers are frozen, only the head is trained, and the small learning rates used for fine-tuning are not suitable. For this reason, the fully frozen models were trained with learning rates of 10^{-4} and 10^{-3} using seed 42 only, and the one with the higher validation score was kept. It should be noted that freezing only lowers the cost of training, namely the backward pass and the optimizer state. During inference, all layers are still executed.

E. Experimental Setup

Table II summarizes the training settings used in all runs. The models were optimized with cross-entropy loss and the AdamW optimizer. A weight decay of 0.01 was applied to all weights except biases and LayerNorm parameters. The learning rate was increased linearly over the first 10% of training steps and then decreased linearly to zero. Gradient clipping was applied with a maximum norm of 1.0, and BF16 mixed precision was used during training. To reduce the amount of padding, the training data were shuffled and then sorted by length within chunks of 50 batches, after which the order of the batches was shuffled again. Each batch was padded only up to the length of its longest sequence. Training ran for up to three epochs. After each epoch, the model was evaluated on the validation set, and the epoch with the highest validation macro-F1 was selected.

Rather than taking a learning rate from previous studies, a small search was carried out, since RoBERTa has been reported to be sensitive to its training settings [8]. The search grid covered the values of 2×10^{-5} and 3×10^{-5} that earlier sarcasm studies used for these two models [2], [6].

A total of 60 runs were conducted in three scenarios. In scenario S1, each of the six model and input combinations was trained with all four learning rates using seed 42, giving 24 runs, and the learning rate with the best validation macro-F1 was chosen. Scenario S2 focused on layer freezing and consisted of 16 runs, including one extra run with the other

TABLE II
TRAINING SETTINGS

Setting	Value
Maximum sequence length	256 tokens
Batch size	32 (evaluation: 128)
Epochs	up to 3, best by validation macro-F1
Loss	cross-entropy
Optimizer	AdamW, weight decay 0.01
Learning rate schedule	10% linear warm-up, linear decay
Gradient clipping	1.0
Classifier dropout	0.1 (RoBERTa), 0.2 (DistilBERT)
Precision	BF16 mixed precision
Learning rate grid (S1)	$\{1, 2, 3, 5\} \times 10^{-5}$
Seeds	42 (S1); 42, 43, 44 (final runs)
Selected learning rate (RoBERTa / DistilBERT)	
Comment-only	1×10^{-5} / 3×10^{-5}
Early fusion	2×10^{-5} / 1×10^{-5}
Late fusion	1×10^{-5} / 1×10^{-5}

learning rate for each partially frozen setting. In scenario S3, the selected configurations were repeated with seeds 43 and 44, resulting in 20 runs. For four of the combinations, the second-best learning rate was also run with these seeds to check whether the choice made with seed 42 remained stable. The selected learning rates were not changed after this check.

All experiments were run on a single NVIDIA GeForce RTX 5070 Ti GPU with 16 GB of memory, using PyTorch 2.11 (CUDA 12.8) and Transformers 5.17. Training time refers to the wall-clock time of one training epoch, not including validation. Memory refers to the peak GPU memory allocated during training. Inference latency refers to the time of the forward pass when the inputs are already on the GPU, so tokenization, padding, and data transfer from the CPU are not included. Latency was measured on 1,000 randomly selected validation examples after 10 warm-up batches, and it is reported in milliseconds per example for batch sizes of 1 and 32. For each configuration, the efficiency values are averaged over the same final runs used to compute its test scores.

F. Evaluation and Statistical Testing

A comment was predicted as sarcastic if its predicted probability was at least 0.5. The reported metrics are accuracy, as well as precision, recall, and F1 for the sarcastic class, together with macro-F1, which is the average F1 score of both classes. Macro-F1 was used as the main metric. It was also the only criterion for choosing epochs and learning rates, so the test set was not involved in either decision.

The test predictions of each run were saved at the end of training but were not evaluated at that point. The 29 final runs were first listed in a manifest file, which was committed to the project repository on 16 October 2026 before any test score was calculated. These runs include the six main configurations with three seeds each, nine partially frozen runs, also with three seeds each, and two fully frozen runs using seed 42. Test results are reported as the mean and sample standard deviation across seeds 42, 43, and 44. The fully frozen models are the exception, as each of them has only one run.

Statistical significance tests were carried out for the main comparisons [9]. Each comparison was made on the predictions of the two models on the same test set and was repeated for every seed. McNemar's test with continuity correction was used to check whether one model was correct more often than the other, based only on the examples where their predictions differed. Since 11 comparisons were made for each seed, the resulting p-values were adjusted using the Holm method. A paired bootstrap with 1,000 resamples of the test set was also used to estimate the 95% confidence interval of the difference in macro-F1. A difference was regarded as significant if its adjusted p-value was below 0.05 and its confidence interval did not include zero, and it was treated as consistent only when this condition was met for all three seeds. A difference that is not significant, however, does not mean that the two configurations perform the same. The 11 comparisons consisted of RoBERTa against DistilBERT for each input configuration, every pair of input configurations within the same model, and two freezing cases: RoBERTa with nine frozen layers against fully fine-tuned DistilBERT, and RoBERTa with six frozen layers against its fully fine-tuned version.

In addition, an exploratory analysis was carried out to find out in which cases the parent comment is most useful. Early fusion and the comment-only configuration were compared on several subgroups of the test set. These subgroups were based on comment length (1–3, 4–10, and more than 10 words), on comments that appear more than once with different labels, and on the confidence of the comment-only RoBERTa model. A comment was considered uncertain when its predicted probability, averaged over the three seeds, was between 0.4 and 0.6. The last grouping used the sentiment of the parent and the comment as scored by VADER [10]. A compound score below -0.05 was treated as negative, and a score above 0.05 as positive. Within each subgroup, the two configurations were compared with McNemar's test for each seed, without correction for multiple comparisons.

III. RESULTS

A. Classification Performance

Table III reports the test scores of the six main configurations together with their computational cost. RoBERTa with early fusion achieved the highest value on every metric, with a macro-F1 of 79.62%. It was followed by RoBERTa with late fusion (79.30%) and RoBERTa with the comment-only input (78.52%). Among the DistilBERT configurations, late fusion and early fusion were almost equal at 77.59% and 77.55%, while the comment-only input reached 76.87%. RoBERTa scored higher than DistilBERT for every input, by 1.65 points for the comment-only input, 2.07 points for early fusion, and 1.71 points for late fusion.

The results were stable across seeds. The standard deviation of accuracy and macro-F1 was at most 0.09 points in every configuration. Precision and recall varied more, especially with early fusion, where the standard deviation of recall reached 2.22 points for RoBERTa and 2.03 points for DistilBERT. The F1 score of these two configurations, however, varied by only about half a point. The test scores were also close to

TABLE III
TEST RESULTS AND COMPUTATIONAL COST OF THE MAIN CONFIGURATIONS

		Test score (%)					Training	Latency (ms)		Memory
Input	Model	Accuracy	Precision	Recall	F1	Macro-F1	(min/epoch)	bs 1	bs 32	(GB)
Comment-only	DistilBERT	76.88 ± 0.08	76.94 ± 0.65	75.80 ± 1.17	76.36 ± 0.29	76.87 ± 0.08	8.56	4.19	0.15	2.63
	RoBERTa	78.53 ± 0.07	78.33 ± 0.36	77.99 ± 0.46	78.16 ± 0.05	78.52 ± 0.06	16.61	8.42	0.31	5.09
Early fusion	DistilBERT	77.58 ± 0.06	78.14 ± 1.11	75.70 ± 2.03	76.88 ± 0.53	77.55 ± 0.09	9.93	4.23	0.32	2.64
	RoBERTa	**79.63** ± 0.05	**79.37** ± 1.31	**79.32** ± 2.22	**79.32** ± 0.48	**79.62** ± 0.06	19.03	8.41	0.61	5.11
Late fusion	DistilBERT	77.60 ± 0.02	77.62 ± 0.07	76.61 ± 0.07	77.12 ± 0.02	77.59 ± 0.02	14.92	7.20	0.42	4.29
	RoBERTa	79.30 ± 0.02	78.82 ± 0.21	79.29 ± 0.39	79.06 ± 0.09	79.30 ± 0.02	28.82	15.06	0.83	8.38

Scores are the mean ± standard deviation over seeds 42, 43, and 44; costs are averaged over the same runs.
Precision, recall, and F1 refer to the sarcastic class. Latency at batch size (bs) 32 is given per example.

the validation scores, with differences in macro-F1 between -0.07 and 0.26 points. Of the 29 final runs, 24 reached their best validation score in the third epoch.

B. Computational Cost

Fig. 1 plots the test macro-F1 against three cost measures. The gap between the two models was similar for all inputs. RoBERTa needed 1.92 to 1.94 times longer to train per epoch, used 1.93 to 1.96 times more GPU memory, and was 1.99 to 2.09 times slower at batch size 1. With the comment-only input, for example, one training epoch took 16.61 minutes for RoBERTa and 8.56 minutes for DistilBERT.

Adding the parent through early fusion increased the training time by a factor of 1.15 for RoBERTa and 1.16 for DistilBERT. The latency at batch size 1 and the peak memory stayed practically the same as with the comment-only input. At batch size 32, on the other hand, the latency per example roughly doubled, for instance from 0.31 to 0.61 ms for RoBERTa. Late fusion was the most expensive input on every measure. Compared with early fusion, it took about 1.5 times longer to train, used 1.62 to 1.64 times more memory, and was 1.70 to 1.79 times slower at batch size 1.

C. Statistical Significance

Nine of the 11 comparisons met the significance criterion for all three seeds. The three differences between RoBERTa and DistilBERT were all significant, and their 95% confidence intervals ranged from 1.34 to 2.33 points across seeds. Both ways of adding the parent improved on the comment-only input for both models. Early fusion added 1.10 points for RoBERTa and 0.69 points for DistilBERT, and late fusion added 0.78 and 0.72 points, respectively. With RoBERTa, late fusion scored 0.32 points lower than early fusion, and this difference was also significant. For DistilBERT, the difference between the two fusion methods was only 0.04 points and not significant (Holm-adjusted $p = 0.97$). In the freezing comparisons, RoBERTa with nine frozen layers outperformed fully fine-tuned DistilBERT by 1.07 points. The drop of 0.24 points caused by freezing six RoBERTa layers was not significant ($p = 0.19$).

D. Effect of the Parent Comment

Fig. 2 shows the macro-F1 gain of early fusion over the comment-only input in several subgroups of the test set. On the whole test set, the gain was 1.10 points for RoBERTa and 0.69 points for DistilBERT. This average hides many changes in individual predictions. For RoBERTa, the parent corrected 6,236 predictions per seed on average, but it also turned 5,119 correct predictions into wrong ones.

The gain was not spread evenly. It was largest for the 10,737 comments on which the comment-only RoBERTa model was uncertain. In this group, macro-F1 rose by 8.18 points for RoBERTa, from about 52% to about 60%, and by 2.44 points for DistilBERT. For comments on which the model was confident, the gain was only 0.25 and 0.47 points. Short comments also gained more than long ones: RoBERTa improved by 2.59 points on comments of one to three words, but by 0.71 points on comments longer than ten words. The 593 comments that occur more than once with different labels (5,886 rows) gave a gain of 2.73 points for RoBERTa, which was significant for every seed. The corresponding gain for DistilBERT (1.51 points) was not significant for one of the three seeds. The sentiment-based groups showed no clear advantage. Pairs with a negative parent and a positive comment gained 0.86 points for RoBERTa, which is lower than the 1.24 points of pairs in which both texts were neutral.

E. Partial Layer Freezing

Fig. 3 shows how freezing affected macro-F1 and training time with early fusion. Freezing six of the 12 RoBERTa layers lowered macro-F1 only from 79.62% to 79.38%, while the training time per epoch fell from 19.03 to 10.51 minutes and the peak memory from 5.11 to 2.70 GB. With nine frozen layers, RoBERTa still reached 78.62%. In this setting it also trained faster and used less memory than fully fine-tuned DistilBERT, with 7.49 against 9.93 minutes per epoch and 1.67 against 2.64 GB. DistilBERT was less tolerant of freezing, as freezing three of its six layers reduced macro-F1 from 77.55% to 76.09%. When every encoder layer was frozen and only the head was trained, both models dropped to about 65% (65.76% for RoBERTa and 65.09% for DistilBERT).

Freezing had little effect on inference speed. Regardless of the number of frozen layers, the latency at batch size 1 stayed between 7.80 and 8.41 ms for RoBERTa and between 3.91 and 4.23 ms for DistilBERT.

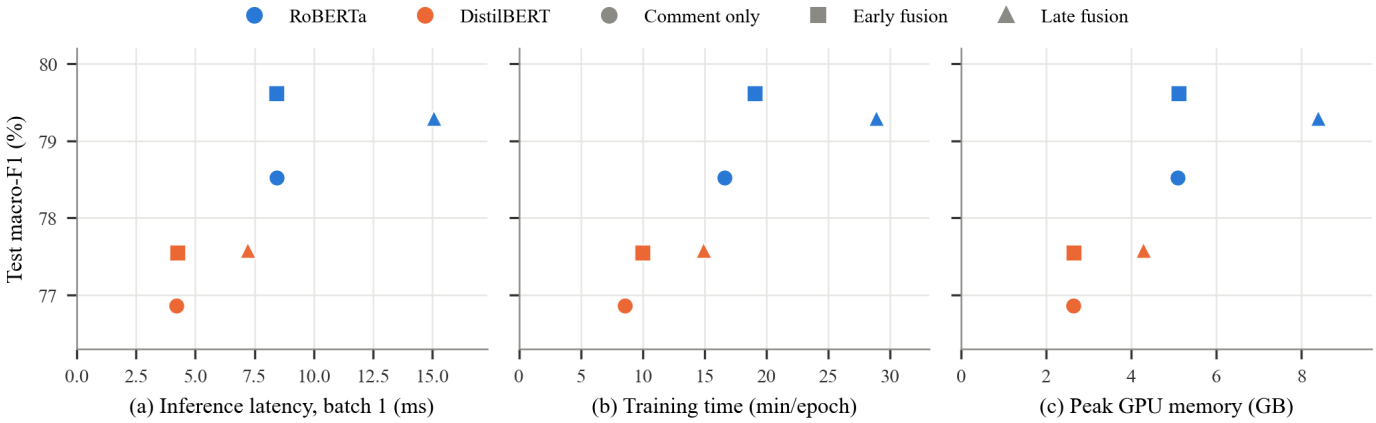


Fig. 1. Test macro-F1 of the six main configurations against (a) inference latency at batch size 1, (b) training time per epoch, and (c) peak GPU memory. Each point is the mean over three seeds, and the error bars show the standard deviation of macro-F1.

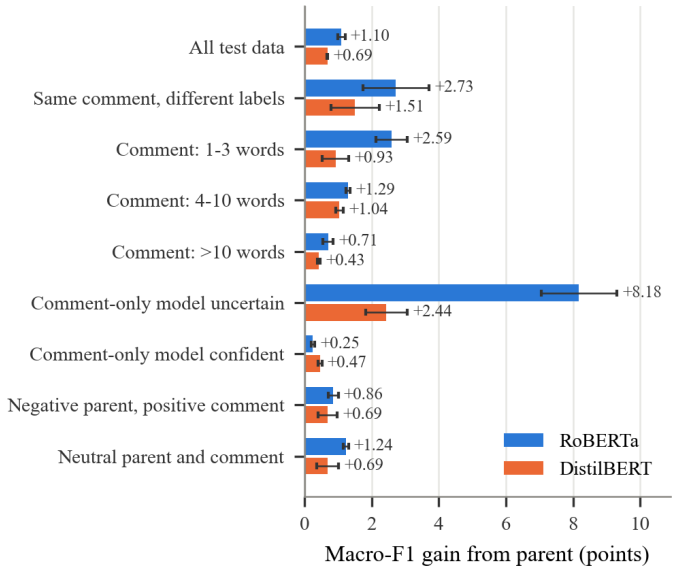


Fig. 2. Macro-F1 gain of early fusion over the comment-only input in subgroups of the test set, as the mean over three seeds with the standard deviation as error bars. The comment-only model is uncertain when its average predicted probability lies between 0.4 and 0.6.

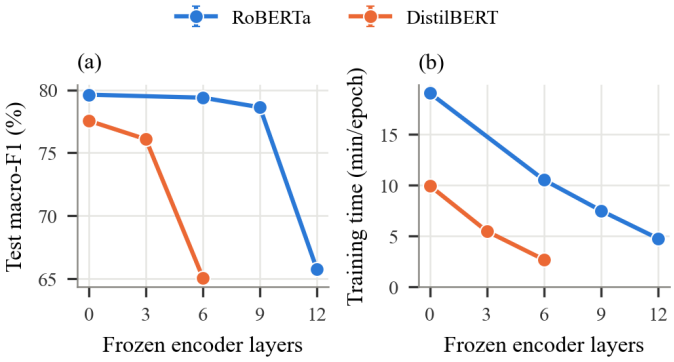


Fig. 3. Effect of freezing the lower encoder layers with early fusion on (a) test macro-F1 and (b) training time per epoch. The settings with all layers frozen were run with one seed, and the others show the mean over three seeds.

IV. Discussion

V. Conclusion

References

[1] M. Khodak, N. Saunshi, and K. Vodrahalli, "A large self-annotated corpus for sarcasm," in *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. Miyazaki, Japan: European Language Resources Association (ELRA), 2018.

[2] P. Dubey, A. Mishra, A. Singh, Murtuza, A. Chanchlani, and P. Dubey, "Efficient transformer architecture for sarcasm detection: A study on compression and performance," *Bulletin of Electrical Engineering and Informatics*, vol. 15, no. 1, pp. 557–567, 2026.

[3] L. Zhang, M. Faseeh, S. S. A. Naqvi, L. Hu, and A. Ghani, "Enhancing sarcasm detection on social media: A comprehensive study using llms and bert with multi-headed attention on sarc," *PLoS ONE*, vol. 20, no. 11, p. e0334120, 2025.

[4] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[6] M. A. Ullah, R. Sultana, S. H. Chowdhury, and F. A. Laila, "Advanced sarcasm detection using transformer-based models: Analyzing variations in formal and informal languages," *Applied Computational Intelligence and Soft Computing*, vol. 2026, p. 8761185, 2026.

[7] T. Shangipour Ataei, S. Javdan, and B. Minaei-Bidgoli, "Applying transformers and aspect-based sentiment analysis approaches on sarcasm detection," in *Proceedings of the Second Workshop on Figurative Language Processing*, 2020, pp. 67–71.

[8] R. Jiménez Castro, J. P. Sánchez-Solís, V. García Jiménez, G. Rivera Zárate, and R. Florencia Juárez, "A comparative study of bert-based models for sarcasm detection in social media texts," *International Journal of Combinatorial Optimization Problems and Informatics*, vol. 17, no. 2, pp. 378–390, 2026.

[9] R. Dror, L. Peled-Cohen, S. Shlomov, and R. Reichart, *Statistical Significance Testing for Natural Language Processing*, ser. Synthesis Lectures on Human Language Technologies. Cham, Switzerland: Springer, 2020.

[10] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 8, no. 1, 2014, pp. 216–225.

[11] R. Dhumpati, A. Sasi, S. J. Basha, S. Vadhri, P. Saroja, S. Alluri, and R. Vatambeti, "Enhancing sarcasm detection in sentiment analysis for

cyberspace safety using advanced deep learning techniques," *Scientific Reports*, vol. 15, p. 22681, 2025.
[12] P. Dubey, P. Dubey, and P. N. Bokoro, "Unpacking sarcasm: A contextual and transformer-based approach for improved detection," *Computers*, vol. 14, no. 3, p. 95, 2025.
[13] V. Grover and H. Banati, "An emoji centric approach to sarcasm detection in online discourse," *Scientific Reports*, vol. 16, p. 3891, 2026.
[14] O. Prakash, "Sarcasm detection for sentiment analysis using deep learning enhancement," *Procedia Computer Science*, vol. 258, pp. 203–212, 2025.